

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 8 SORTING 2 (SHELL, QUICK, DAN MERGE SORT)



Disusun Oleh :

DIKA GIOWANDA

NIM 2311533025

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

SHERLY SUKMADIRA PUTRI

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, MEI 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas limpahan rahmat, hidayah, dan karunia-Nya sehingga laporan praktikum Struktur Data tentang implementasi algoritma *Sorting 2* (*Shell Sort*, *Quick Sort*, dan *Merge Sort*) menggunakan bahasa pemrograman Java ini dapat diselesaikan dengan baik dan tepat waktu. Sholawat serta salam senantiasa tercurah kepada Nabi Muhammad SAW, keluarga, sahabat, dan seluruh pengikutnya hingga akhir zaman.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban, refleksi, serta dokumentasi atas kegiatan praktikum yang telah dilaksanakan, dengan fokus pada penerapan konsep algoritma pengurutan atau sorting dalam struktur data. *Shell Sort*, *Quick Sort*, dan *Merge Sort* merupakan algoritma *sorting* yang penting untuk dipelajari karena memiliki karakteristik dan kompleksitas waktu yang berbeda-beda dalam mengurutkan data. Dalam praktikum ini, penulis mempelajari implementasi ketiga algoritma tersebut menggunakan bahasa pemrograman Java, memahami cara kerja masing-masing algoritma, menganalisis kompleksitas waktu dan ruang, serta membandingkan efisiensi dari ketiga metode *sorting* tersebut dalam mengolah data.

Penyusunan laporan ini tidak mungkin terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, penulis mengucapkan terima kasih yang tulus kepada dosen pembimbing yang telah memberikan arahan, bimbingan, dan masukan yang berharga selama proses praktikum, serta kepada rekan-rekan mahasiswa yang turut berkontribusi dalam diskusi dan kolaborasi.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca, khususnya dalam memahami konsep dan implementasi algoritma *Shell Sort*, *Quick Sort*, dan *Merge Sort* dalam bahasa pemrograman Java.

Akhir kata, penulis berharap semoga ilmu yang diperoleh melalui praktikum ini dapat bermanfaat dan diterapkan dengan baik.

Padang, 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI.....	iii
BAB 1 PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Tujuan Praktikum.....	1
1.3 Persyaratan Praktikum.....	1
1.4 Tempat dan Waktu Praktikum.....	2
1.5 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN PRAKTIKUM.....	3
2.1 Pengertian Pemrograman Sorting 2.....	3
2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 8.....	4
BAB 3 PENUTUP.....	23
3.1 Kesimpulan.....	23
3.2 Saran.....	23
DAFTAR PUSTAKA.....	24
LAPORAN TUGAS.....	25

BAB 1

PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan konsep pemrograman Java. Kegiatan ini tidak hanya menekankan pada penguasaan teori, tetapi juga pada latihan penyusunan kode program, pengujian, hingga analisis hasil eksekusi. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual dengan kemampuan teknis pemrograman.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut :

1. Membantu mahasiswa memahami konsep dasar pemrograman Struktur Data melalui penerapan langsung.
2. Melatih kemampuan menulis, mengompilasi, dan mengeksekusi program dengan mengikuti aturan sintaksis Java.
3. Memahami tipe data dasar hingga kompleks, termasuk konsep penyimpanan data.
4. Mampu menerapkan struktur data linear (*Linked List, Stack, Queue*) dan dalam bahasa pemrograman
5. Membiasakan mahasiswa bekerja sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
6. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.
7. Mengetahui dan mengaplikasikan *ADT, ArrayList, Stack, Queue, Single Linked List, Double Linked List, Sorting 1 (Insertion, selection, bubble), Sorting 2 (shell, quick, merge), dan Searching (BFS dan DFS)*.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang sudah terpasang *Java Development Kit (JDK)* dan *Integrated Development Environment (IDE)* atau *Eclips* yang direkomendasikan.
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

1.5 Manfaat Praktikum

Manfaat praktikum Struktur Data meliputi kemampuan memahami cara menyimpan data secara terstruktur, rapi, dan efisien, sehingga mengurangi penggunaan memori dan mempercepat akses data. Meningkatkan efisiensi penyimpanan dan manipulasi data, mengoptimalkan kecepatan algoritma, serta mempermudah penyelesaian masalah pemrograman yang kompleks. Selain itu, praktikum ini juga membantu membangun fondasi untuk mengembangkan aplikasi lintas platform yang lebih kompleks di berbagai bidang teknologi.

BAB 2

PEMBAHASAN PRAKTIKUM

2.1 Pengertian Pemrograman Sorting 2

Sorting 2 dalam praktikum struktur data Java adalah materi lanjutan yang mempelajari algoritma pengurutan yang lebih efisien dan kompleks (lanjutan dari Simple Sorting seperti *Bubble/Selection/Insertion sort*). Ketiga algoritma ini dirancang untuk mengurutkan kumpulan data dengan sangat cepat. Komponen utama *Shell sort*, *Quick sort*, dan *Merge sort* dalam praktikum struktur data Java berpusat pada optimalisasi efisiensi pengurutan (*kompleksitas* $O(n \log n)$). Algoritma ini memodifikasi metode dasar seperti insertion dan menerapkan konsep *Divide and Conquer* (memecah dan menaklukkan) menggunakan fungsi rekursif.

Berikut adalah penjelasan ringkas, terstruktur, dan mudah dipahami untuk masing-masing algoritma:

1. *Shell Sort* Merupakan pengembangan dari *Insertion Sort*. Algoritma ini mengurutkan data dengan membandingkan elemen yang memiliki jarak (*interval/gap*) tertentu, kemudian secara bertahap memperkecil jarak tersebut hingga bernilai 1.
 - Cara Kerja: Membagi *array* besar menjadi beberapa *sub-array* yang lebih kecil, mengurutkan *sub-array* tersebut, lalu menggabungkannya kembali.
 - Kelebihan: Sangat cepat untuk mengurutkan sebagian data yang hampir terurut dibandingkan algoritma sederhana.
2. *Quick Sort* Algoritma ini sering disebut sebagai salah satu algoritma pengurutan tercepat. Menggunakan pendekatan *Divide and Conquer* (membagi dan menaklukkan) berbasis *pivot* (titik tengah/acuan).
 - Cara Kerja: Pilih satu elemen sebagai *pivot*. Pindahkan semua elemen yang lebih kecil ke kiri *pivot*, dan yang lebih besar ke kanan *pivot* (proses *partitioning*). Lakukan proses ini secara

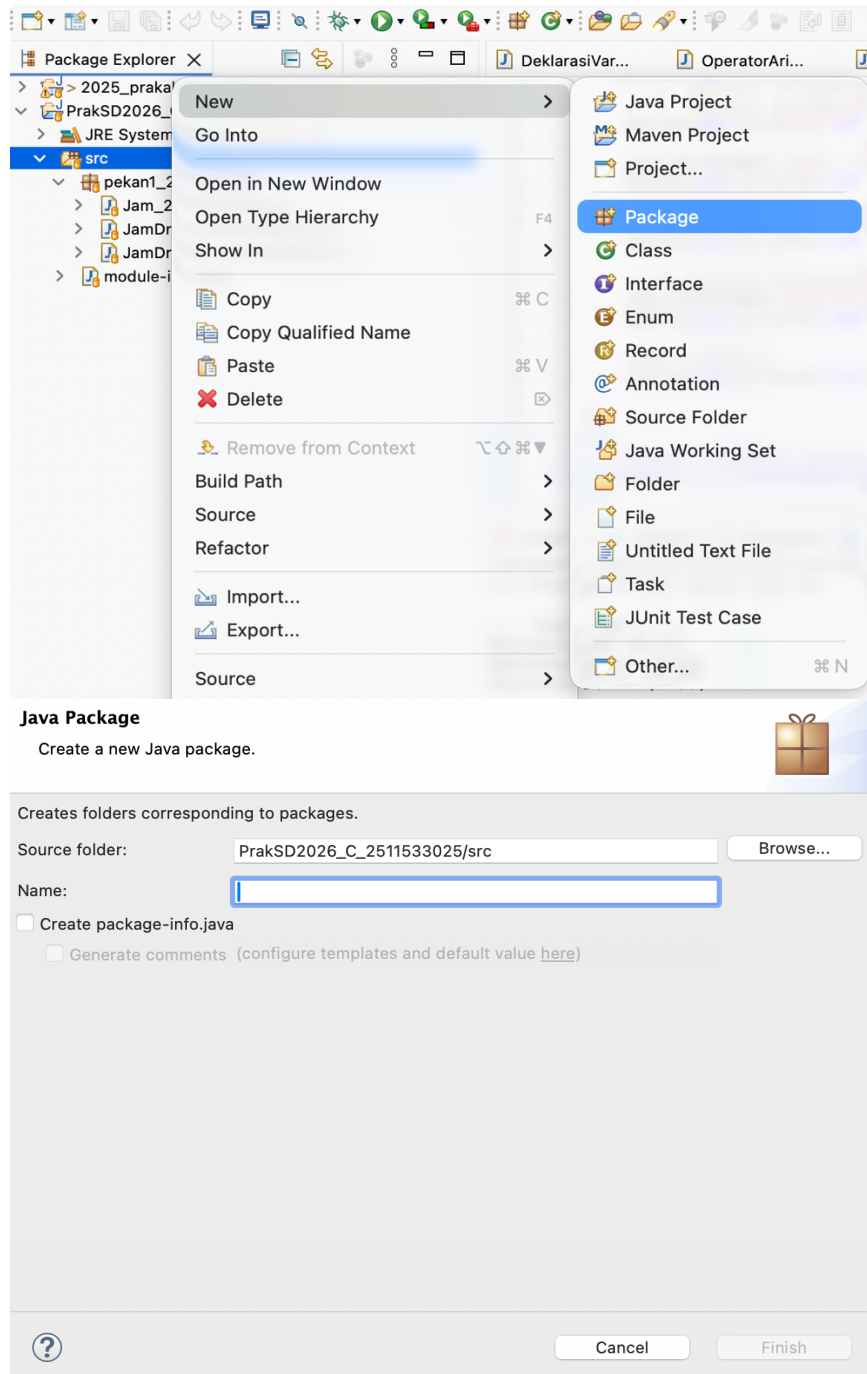
rekursif (berulang) pada *sub-array* kiri dan kanan sampai seluruhnya terurut.

- Kelebihan: Sangat efisien untuk himpunan data yang sangat besar.
3. Merge Sort Algoritma stabil dan sangat terstruktur yang juga menggunakan metode *Divide and Conquer*.
- Cara Kerja: *Divide*: Memecah (membagi) kumpulan data secara terus-menerus menjadi dua bagian sama besar sampai setiap bagian hanya tersisa satu elemen. *Conquer/Merge*: Menggabungkan kembali potongan-potongan tersebut secara berurutan hingga membentuk satu array utuh yang rapi.
 - Kelebihan: Waktu eksekusi yang konsisten (efisien untuk data terburuk/terbaik) dan sangat stabil.

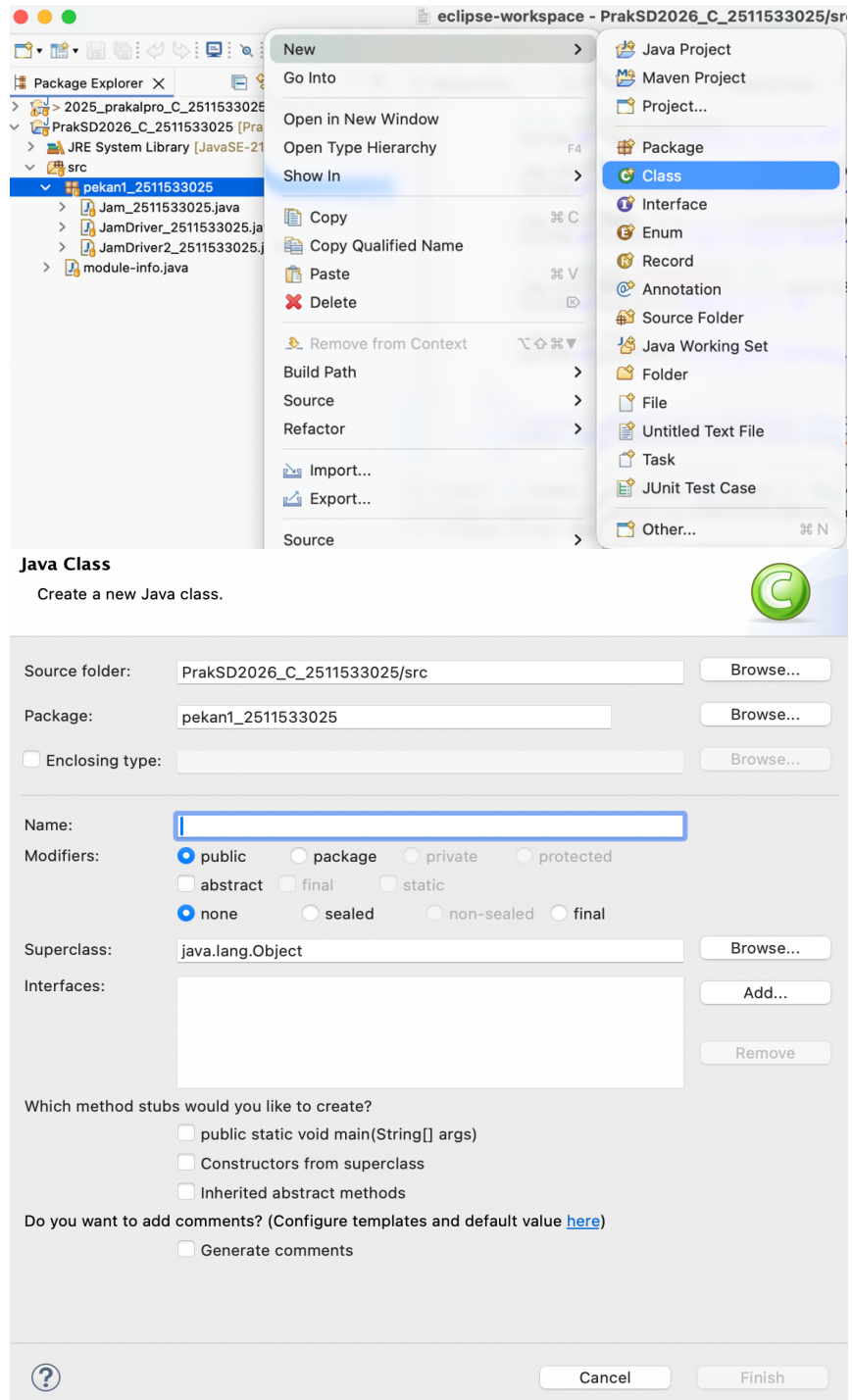
2.2 Langkah-Langkah Pengerjaan Praktikum Pekan 8

Sebelum ke langkah-langkah pengerjaan contoh proyek, saya akan menjelaskan terlebih dahulu gimana caranya menambahkan proyek nya terlebih dahulu :

1. Klik kanan pada folder src, pilih new, pilih package. Nanti akan muncul halaman Java Package, cukup masukan nama, nama wajib no spasi. Klik finish.



2. Klik kanan pada folder package, pilih new, pilih class. Nanti akan muncul halaman Java Class, cukup masukan nama class sesuai proyek yang akan dibuat, nama wajib no spasi. Klik finish.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru.

- A. Baiklah karena proses menambahkan proyek sudah dijelaskan, selanjutnya langkah-langkah pengerjaan proyek pekan 7:

a. Class ShellSort_2511533025

1. Kode ini untuk mengurutkan angka dalam sebuah array secara efisien. Cara kerjanya adalah dengan membandingkan dan menukar elemen yang memiliki jarak tertentu (gap), dimulai dari jarak yang besar (setengah dari panjang array) kemudian secara bertahap diperkecil hingga menjadi satu. Dengan metode ini, elemen-elemen yang letaknya berjauhan dapat berpindah ke posisi yang tepat lebih cepat dibandingkan algoritma pengurutan sederhana lainnya, hingga akhirnya seluruh array terurut secara sempurna.

```
1 package Pekan8_2511533025;
2
3 public class ShellSort_2511533025 {
4     public static void shellSort(int[] A_3025) {
5         int n_3025 = A_3025.length;
6         int gap_3025 = n_3025 / 2;
7         while (gap_3025 > 0) {
8             for (int i_3025 = gap_3025; i_3025 < n_3025; i_3025++) {
9                 int temp_3025 = A_3025[i_3025];
10                int j_3025 = i_3025;
11                while (j_3025 >= gap_3025 && A_3025[j_3025 - gap_3025] > temp_3025) {
12                    A_3025[j_3025] = A_3025[j_3025 - gap_3025];
13                    j_3025 = j_3025 - gap_3025;
14                }
15                A_3025[j_3025] = temp_3025;
16            }
17            gap_3025 = gap_3025 / 2;
18        }
19    }
20 }
```

2. Kode ini mendemonstrasikan program dimulai dengan menyiapkan daftar angka acak, menampilkan urutan angka tersebut ke layar, menjalankan proses pengurutan, dan diakhiri dengan mencetak kembali hasil array yang telah terurut sehingga pengguna dapat melihat perbedaan sebelum dan sesudah proses pengurutan dilakukan.

```
20 public static void main(String[] args) {
21     int[] data_3025 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
22     System.out.print("Sebelum: ");
23     printArray(data_3025);
24     shellSort(data_3025);
25     System.out.print("Sesudah (Shell Sort): ");
26     printArray(data_3025);
27 }
28 public static void printArray(int[] arr_3025) {
29     for (int i_3025 : arr_3025) System.out.print(i_3025 + " ");
30     System.out.println();
31 }
32 }
```

b. Class QuickSort_2511533025

1. Kode ini merupakan sebuah fungsi bernama swap yang berguna untuk menukar posisi dua elemen di dalam sebuah array. Secara sederhana, fungsi ini bekerja dengan menyimpan nilai pertama ke tempat penampungan sementara (temp), mengisi posisi

pertama dengan nilai kedua, lalu memindahkan nilai dari penampungan tadi ke posisi kedua agar kedua angka tersebut bertukar tempat tanpa ada data yang hilang.

```

1 package Pekan8_2511533025;
2 public class QuickSort_2511533025 {
3     static void swap(int[] arr_3025, int i_3025, int j_3025)
4     {
5         int temp_3025 = arr_3025[i_3025];
6         arr_3025[i_3025] = arr_3025[j_3025];
7         arr_3025[j_3025] = temp_3025;
8     }

```

2. Kode fungsi metode Median-of-Three untuk memilih nilai pivot yang lebih optimal dalam algoritma pengurutan seperti Quicksort. Fungsi ini membandingkan tiga elemen dalam array—yaitu elemen di posisi awal, tengah, dan akhir—lalu mengurutkan ketiganya agar nilai yang berada di tengah merupakan median dari ketiga angka tersebut. Terakhir, nilai median tersebut dipindahkan ke posisi akhir (high) untuk digunakan sebagai acuan pembagi data, yang bertujuan untuk meningkatkan efisiensi algoritma dan menghindari performa buruk pada data yang sudah hampir terurut.

```

9 // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
10 static void medianOfThree(int[] arr_3025, int low_3025, int high_3025)
11 {
12     int mid_3025 = low_3025 + (high_3025 - low_3025) / 2;
13
14     // Urutkan elemen low, mid, dan high
15     if (arr_3025[low_3025] > arr_3025[mid_3025]) {
16         swap(arr_3025, low_3025, mid_3025);
17     }
18     if (arr_3025[low_3025] > arr_3025[high_3025]) {
19         swap(arr_3025, low_3025, high_3025);
20     }
21     if (arr_3025[mid_3025] > arr_3025[high_3025]) {
22         swap(arr_3025, mid_3025, high_3025);
23     }
24     swap(arr_3025, mid_3025, high_3025);
25 }

```

3. Kode ini merupakan implementasi fungsi partition dalam algoritma pengurutan QuickSort yang bertujuan untuk mengatur ulang elemen dalam array di sekitar sebuah nilai pusat yang disebut pivot. Fungsi ini menggunakan metode median of three untuk memilih pivot yang lebih optimal agar pembagian data lebih seimbang, kemudian memindahkan semua elemen yang nilainya lebih kecil dari pivot ke sisi kiri dan yang lebih besar ke sisi kanan. Setelah proses pemindahan selesai, pivot diletakkan di posisi yang tepat di tengah-tengah kedua kelompok tersebut, lalu fungsi mengembalikan indeks posisi pivot tersebut sebagai titik pembagi untuk tahap pengurutan berikutnya.


```

26 static int partition(int[] arr_3025, int low_3025, int high_3025)
27 {
28     // Panggil fungsi medianOfThree sebelum menentukan pivot
29     medianOfThree(arr_3025, low_3025, high_3025);
30
31     int pivot = arr_3025[high_3025]; // Sekarang arr[high] sudah berisi nilai median
32     int i_3025 = (low_3025 - 1);
33
34     for (int j_3025 = low_3025; j_3025 <= high_3025 - 1; j_3025++) {
35         // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
36         if (arr_3025[j_3025] < pivot) {
37             // Increment indeks elemen yang lebih kecil
38             i_3025++;
39             swap(arr_3025, i_3025, j_3025);
40         }
41     }
42     swap(arr_3025, i_3025 + 1, high_3025);
43     return (i_3025 + 1);
44 }

```

4. Kode ini fungsi quickSort bekerja secara rekursif dengan membagi array menjadi bagian-bagian yang lebih kecil berdasarkan titik pembagi (pivot) hingga seluruh elemen tersusun urut, sedangkan fungsi printArr bertugas mencetak setiap elemen array ke layar satu per satu agar hasil pengurutan tersebut dapat dilihat oleh pengguna.

```

45 static void quickSort(int[] arr_3025, int low_3025, int high_3025)
46 {
47     if (low_3025 < high_3025) {
48         int pi_3025 = partition(arr_3025, low_3025, high_3025);
49         quickSort(arr_3025, low_3025, pi_3025 - 1);
50         quickSort(arr_3025, pi_3025 + 1, high_3025);
51     }
52 }
53 public static void printArr(int[] arr_3025)
54 {
55     for (int i_3025 = 0; i_3025 < arr_3025.length; i_3025++) {
56         System.out.print(arr_3025[i_3025] + " ");
57     }
58     System.out.println();
59 }

```

5. Kode tersebut adalah fungsi utama program yang bertujuan untuk mengurutkan sekumpulan angka dalam sebuah daftar (array) menggunakan metode QuickSort. Program ini pertama-tama menyiapkan data angka acak, menampilkan data tersebut ke layar, lalu menjalankan proses pengurutan agar angka-angka tersusun rapi dari yang terkecil hingga terbesar, dan terakhir mencetak kembali hasilnya untuk menunjukkan bahwa data telah berhasil diurutkan.

```

60 public static void main(String[] args)
61 {
62     int[] arr_3025 = { 10, 7, 8, 9, 1, 5 };
63     int N_3025 = arr_3025.length;
64     System.out.print("Data sebelum diurutkan: ");
65     printArr(arr_3025);
66
67     quickSort(arr_3025, 0, N_3025 - 1);
68
69     System.out.print("Data Terurut quicksort: ");
70     printArr(arr_3025);
71 }
72 }

```


c. Class MergeSort_2511533025

1. Kode ini untuk metode merge dalam algoritma Merge Sort yang berfungsi untuk menyiapkan proses penggabungan dua bagian array yang terpisah. Secara sederhana, kode ini menghitung ukuran dua sub-array sementara, membuat wadah baru untuk keduanya, lalu menyalin data dari array utama ke dalam sub-array kiri dan kanan tersebut. Langkah ini dilakukan agar data dapat disimpan sementara sebelum nantinya dibandingkan dan disusun kembali dalam urutan yang benar.

```
1 package Pekan8_2511533025;
2 public class MergeSort_2511533025 {
3     void merge(int arr[], int l_3025, int m_3025, int r_3025) {
4         // Find sizes of two subarrays to be merged
5         int n1_3025 = m_3025 - l_3025 + 1;
6         int n2_3025 = r_3025 - m_3025;
7         /* Create temp arrays */
8         int L_3025[] = new int[n1_3025];
9         int R_3025[] = new int[n2_3025];
10        /* Copy data to temp arrays */
11        for (int i_3025 = 0; i_3025 < n1_3025; ++i_3025)
12            L_3025[i_3025] = arr[l_3025 + i_3025];
13        for (int j_3025 = 0; j_3025 < n2_3025; ++j_3025)
14            R_3025[j_3025] = arr[m_3025 + 1 + j_3025];
15        int i_3025 = 0, j_3025 = 0;
```

2. Kode ini untuk menggabungkan dua sub-array yang sudah terurut menjadi satu array tunggal yang tetap urut. Proses ini dilakukan dengan membandingkan elemen-elemen dari kedua sub-array (kiri dan kanan) satu per satu, lalu mengambil nilai yang lebih kecil untuk dimasukkan ke array utama. Setelah proses perbandingan selesai, kode tersebut juga memastikan bahwa jika masih ada elemen yang tersisa di salah satu sub-array, elemen tersebut akan langsung disalin ke posisi akhir array utama agar urutannya tetap terjaga secara keseluruhan.

```
16        // Initial index of merged subarray array
17        int k_3025 = l_3025;
18        while (i_3025 < n1_3025 && j_3025 < n2_3025) {
19            if (L_3025[i_3025] <= R_3025[j_3025]) {
20                arr[k_3025] = L_3025[i_3025];
21                i_3025++;
22            } else {
23                arr[k_3025] = R_3025[j_3025];
24                j_3025++;
25            }
26            k_3025++;
27        }
28        /* Copy remaining elements of L[] if any */
29        while (i_3025 < n1_3025) {
30            arr[k_3025] = L_3025[i_3025];
31            i_3025++;
32            k_3025++;
33        }
34        /* Copy remaining elements of R[] if any */
35        while (j_3025 < n2_3025) {
36            arr[k_3025] = R_3025[j_3025];
37            j_3025++;
38            k_3025++;
39        }
40    }
```

3. Kode ini merupakan implementasi dari algoritma Merge Sort yang menggunakan pendekatan divide and conquer (bagi dan taklukkan) untuk mengurutkan data. Secara sederhana, fungsi ini bekerja dengan membagi sebuah daftar (array) menjadi dua bagian secara terus-menerus hingga menjadi elemen-elemen kecil, mengurutkan bagian-bagian tersebut secara terpisah melalui rekursi, dan akhirnya menggabungkannya kembali dalam urutan yang benar menggunakan fungsi merge.

```
41 void sort(int arr_3025[], int l_3025, int r_3025) {
42     if (l_3025 < r_3025) {
43         // Find the middle point
44         int m_3025 = (l_3025 + r_3025) / 2;
45         // Sort first and second halves
46         sort(arr_3025, l_3025, m_3025);
47         sort(arr_3025, m_3025 + 1, r_3025);
48         // Merge the sorted halves
49         merge(arr_3025, l_3025, m_3025, r_3025);
50     }
51 }
```

4. Kode ini fungsi printArray tersebut digunakan untuk menampilkan atau mencetak semua elemen yang ada di dalam sebuah array angka ke layar secara berurutan. Kode ini bekerja dengan cara mencari tahu panjang array terlebih dahulu, lalu menggunakan perulangan (looping) untuk mencetak setiap angka satu per satu dalam satu baris yang dipisahkan oleh spasi. Setelah semua isi array selesai ditampilkan.

```
52 /* A utility function to print array of size n */
53 static void printArray(int arr_3025[]) {
54     int n_3025 = arr_3025.length;
55     for (int i_3025 = 0; i_3025 < n_3025; ++i_3025)
56         System.out.print(arr_3025[i_3025] + " ");
57     System.out.println();
58 }
```

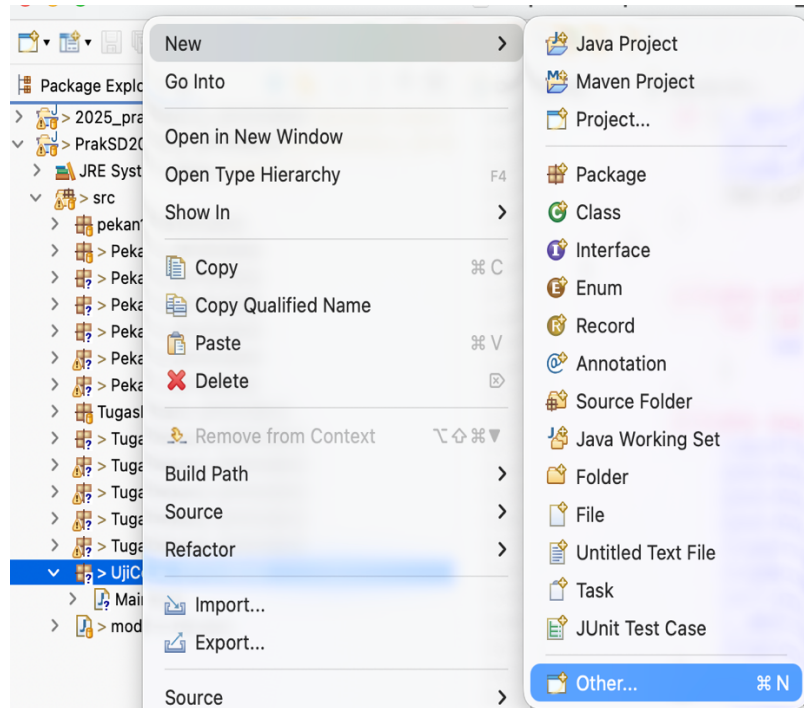
5. Kode ini merupakan fungsi utama (main) yang bertujuan untuk mendemonstrasikan proses pengurutan angka menggunakan algoritma Merge Sort. Program dimulai dengan mendefinisikan sebuah kumpulan angka (array) acak, kemudian menampilkannya ke layar sebelum diurutkan. Selanjutnya, program membuat objek dari kelas pengurutan dan memanggil metode sort untuk menyusun angka-angka tersebut dari yang terkecil ke terbesar, lalu diakhiri dengan mencetak.

```
59 public static void main(String args[]) {
60     int arr_3025[] = { 12, 11, 13, 5, 6, 7 };
61     System.out.println("Sebelum terurut");
62     printArray(arr_3025);
63     MergeSort_2511533025 ob_3025 = new MergeSort_2511533025();
64     ob_3025.sort(arr_3025, 0, arr_3025.length - 1);
65     System.out.println("\nSesudah Terurut menggunakan merge Sort");
66     printArray(arr_3025);
67 }
68 }
```

d. Class BubbleGUI_2511533025

Class yang ini berbeda dengan class sebelumnya karena tampilan outputnya berbentuk GUI.

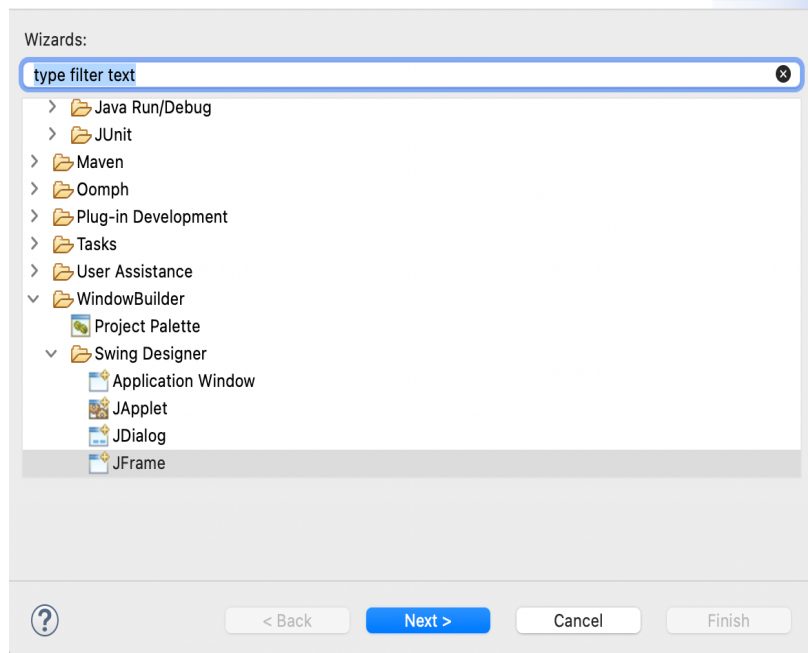
1. Buat class terlebih dahulu, caranya klik kanan pada package yang ingin digunakan, pilih new, terus pilih Other.



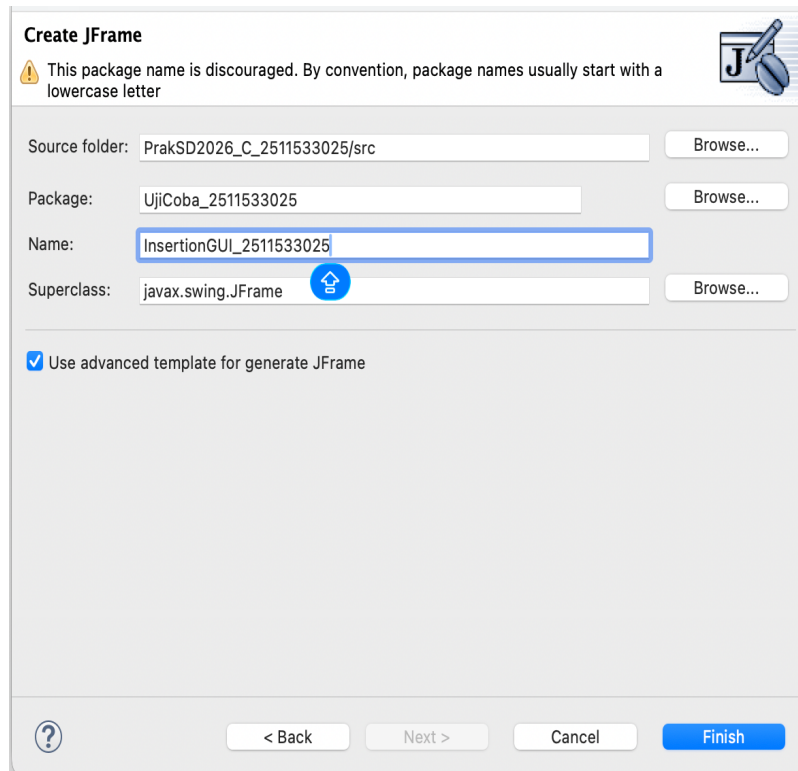
2. Kemudian pilih WindowBuilder, lalu pilih Swing Designer, terus pilih JFrame. Klik Next.

Select a wizard

Create an empty JFrame



3. Terakhir masukan nama pada bagian name, klik Finish. Lanjut masukan kode sesuai program yang ingin dibuat.



4. Kode ini untuk instruksi import dalam bahasa pemrograman Java yang berfungsi untuk memanggil berbagai pustaka

(library) guna membangun antarmuka grafis (GUI). Dengan perintah-perintah ini, program dapat menggunakan komponen visual siap pakai seperti jendela aplikasi (JFrame), tombol (JButton), label teks, kotak input, hingga pengaturan tata letak (layout) agar tampilan aplikasi terlihat rapi dan interaktif.

```

1 package Pekan8_2511533025;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.EventQueue;
7 import java.awt.FlowLayout;
8 import java.awt.Font;
9 import javax.swing.BorderFactory;
10 import javax.swing.JButton;
11 import javax.swing.JFrame;
12 import javax.swing.JLabel;
13 import javax.swing.JOptionPane;
14 import javax.swing.JPanel;
15 import javax.swing.JScrollPane;
16 import javax.swing.JTextArea;
17 import javax.swing.JTextField;
18 import javax.swing.SwingConstants;
19 import javax.swing.SwingUtilities;
20 import javax.swing.border.EmptyBorder;
21 import Pekan7_2511533025.InsertionGUI_2511533025;
22

```

5. Kode ini mendefinisikan struktur dasar untuk sebuah aplikasi antarmuka pengguna (GUI) bernama BubbleGUI yang menggunakan pustaka Swing (ditandai dengan JFrame). Di dalamnya terdapat berbagai variabel pendukung untuk membuat simulasi algoritma pengurutan, seperti array angka, label dan panel untuk visualisasi, serta komponen interaktif seperti tombol (step, reset, set), bidang teks untuk input, dan area teks untuk catatan langkah. Selain itu, kode ini menyiapkan variabel kontrol seperti `i_3025`, `j_3025`, dan `stepCount_3025` untuk melacak proses pengurutan data secara bertahap.

```

23 public class BubbleGUI_2511533025 extends JFrame {
24
25     private static final long serialVersionUID = 1L;
26     private int[] array_3025;
27     private JLabel[] labelArray_3025;
28     private JButton stepButton_3025, resetButton_3025, setButton_3025;
29     private JTextField inputField_3025;
30     private JPanel panelArray_3025;
31     private JTextArea stepArea_3025;
32
33     private int i_3025 = 1, j_3025;
34     private boolean sorting_3025 = false;
35     private int stepCount_3025 = 1;
36

```

6. Kode ini merupakan konstruktor yang berfungsi untuk menginisialisasi jendela utama aplikasi (GUI) Java Swing. Di dalamnya, dilakukan pengaturan dasar seperti memberikan judul jendela, menentukan ukuran (772x400 piksel), mengatur agar aplikasi otomatis berhenti saat jendela ditutup, memosisikan jendela tepat di tengah layar, serta menetapkan tata letak (layout) komponen menggunakan BorderLayout.


```

37  /**
38   * Create the frame.
39   */
40  public BubbleGUI_2511533025() {
41      setTitle("Bubble Sort Langkah per Langkah");
42      setSize(772, 400);
43      setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
44      setLocationRelativeTo(null);
45      setLayout(new BorderLayout());
46  }

```

7. Kode ini berfungsi untuk membuat sebuah panel input dalam antarmuka grafis (GUI) Java menggunakan library Swing.

```

47  // Panel input
48  JPanel inputPanel_3025 = new JPanel(new FlowLayout());
49  inputField_3025 = new JTextField(30);
50  setButton_3025 = new JButton("Set Array");
51  inputPanel_3025.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
52  inputPanel_3025.add(inputField_3025);
53  inputPanel_3025.add(setButton_3025);
54

```

8. Kode ini berfungsi untuk membuat dan mengatur tata letak sebuah panel visual (objek JPanel).

```

55  // Panel array visual
56  panelArray_3025 = new JPanel();
57  panelArray_3025.setLayout(new FlowLayout());
58

```

9. Kode ini untuk membangun antarmuka pengguna (GUI) sederhana yang terdiri dari dua bagian utama: panel kontrol dan area tampilan teks. Di bagian panel kontrol, kode membuat dua tombol yaitu "Langkah Selanjutnya" (yang diatur agar tidak bisa diklik saat awal) dan tombol "Reset". Sedangkan di bagian area teks, kode menyiapkan tempat untuk menampilkan catatan atau log langkah-langkah dengan pengaturan huruf tertentu, sifatnya yang tidak bisa diubah oleh pengguna (read-only), serta dilengkapi dengan bilah gulir (scroll pane) agar teks yang panjang tetap mudah dibaca.

```

59  // Panel kontrol
60  JPanel controlPanel_3025 = new JPanel();
61  stepButton_3025 = new JButton("Langkah Selanjutnya");
62  resetButton_3025 = new JButton("Reset");
63  stepButton_3025.setEnabled(false);
64  controlPanel_3025.add(stepButton_3025);
65  controlPanel_3025.add(resetButton_3025);
66  // Area teks untuk log langkah-langkah
67  stepArea_3025 = new JTextArea(8, 60);
68  stepArea_3025.setEditable(false);
69  stepArea_3025.setFont(new Font("Monospaced", Font.PLAIN, 14));
70  JScrollPane scrollPane_3025 = new JScrollPane(stepArea_3025);
71

```

10. Kode ini berfungsi untuk mengatur tata letak (layout) komponen antarmuka pengguna di dalam sebuah jendela utama (frame). Dengan menggunakan sistem BorderLayout, kode ini menempatkan panel input di bagian atas (NORTH), panel utama atau array di bagian tengah (CENTER), panel kontrol di bagian

bawah (SOUTH), dan panel gulir di sisi kanan (EAST). Intinya, kode ini menyusun berbagai elemen visual program agar terorganisir dengan rapi sesuai posisi mata angin yang ditentukan.

```

72      // Tambahkan panel ke frame
73      add(inputPanel_3025, BorderLayout.NORTH);
74      add(panelArray_3025, BorderLayout.CENTER);
75      add(controlPanel_3025, BorderLayout.SOUTH);
76      add(scrollPane_3025, BorderLayout.EAST);
77

```

11. Kode ini berfungsi untuk memberikan instruksi pada tiga tombol di antarmuka program (GUI) menggunakan bahasa Java. Secara sederhana, baris-baris ini mengatur agar program menjalankan aksi tertentu saat tombol diklik: tombol Set Array akan mengambil data input, tombol Langkah Selanjutnya akan menjalankan urutan proses berikutnya, dan tombol Reset akan mengembalikan kondisi program ke awal. Semua instruksi ini menggunakan lambda expression (e -> ...) untuk memanggil fungsi-fungsi yang telah ditentukan sebelumnya secara singkat dan efisien.

```

78      // Event Set Array
79      setButton_3025.addActionListener(e -> setArrayFromInput());
80
81      // Event Langkah Selanjutnya
82      stepButton_3025.addActionListener(e -> performStep());
83
84      // Event Reset
85      resetButton_3025.addActionListener(e -> reset());
86  }

```

12. Kode ini berfungsi untuk mengambil teks masukan dari sebuah kolom input, membersihkan spasi di awal dan akhir, lalu memecahnya menjadi bagian-bagian terpisah berdasarkan tanda koma. Setiap bagian teks tersebut kemudian diubah menjadi angka bulat (integer) dan disimpan ke dalam sebuah array. Jika pengguna memasukkan karakter yang bukan angka, program akan menangkap kesalahan tersebut dan menampilkan pesan peringatan melalui kotak dialog agar pengguna hanya memasukkan angka yang dipisahkan oleh koma.

```

87  private void setArrayFromInput() {
88      String text_3025 = inputField_3025.getText().trim();
89      if (text_3025.isEmpty()) return;
90      String[] parts_3025 = text_3025.split(",");
91      array_3025 = new int[parts_3025.length];
92      try {
93          for (int k_3025 = 0; k_3025 < parts_3025.length; k_3025++) {
94              array_3025[k_3025] = Integer.parseInt(parts_3025[k_3025].trim());
95          }
96      } catch (NumberFormatException e) {
97          JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
98              + "yang dipisahkan koma!", "Error", JOptionPane.ERROR_MESSAGE);
99      }
100  }

```

13. Kode Java Swing ini berfungsi untuk mereset dan menampilkan visualisasi array baru ke dalam antarmuka pengguna (GUI). Secara teknis, kode ini menginisialisasi ulang variabel kontrol, membersihkan panel tampilan (panelArray_3025), lalu menggunakan perulangan for untuk membuat serangkaian label grafis (JLabel) berdasarkan isi elemen array. Setiap label diformat dengan desain khusus seperti bingkai hitam, latar belakang putih, dan teks tebal sebelum akhirnya dimasukkan kembali ke dalam panel dan diperbarui tampilannya.

```
101     i_3025 = 0;
102     j_3025 = 0;
103     stepCount_3025 = 1;
104     sorting_3025 = true;
105     stepButton_3025.setEnabled(true);
106     stepArea_3025.setText("");
107     panelArray_3025.removeAll();
108     labelArray_3025 = new JLabel[array_3025.length];
109     for (int k_3025 = 0; k_3025 < array_3025.length; k_3025++) {
110         labelArray_3025[k_3025] = new JLabel(String.valueOf(array_3025[k_3025]));
111         labelArray_3025[k_3025].setFont(new Font("Arial", Font.BOLD, 24));
112         labelArray_3025[k_3025].setOpaque(true);
113         labelArray_3025[k_3025].setBackground(Color.WHITE);
114         labelArray_3025[k_3025].setBorder(BorderFactory.createLineBorder(Color.BLACK));
115         labelArray_3025[k_3025].setPreferredSize(new Dimension(50, 50));
116         labelArray_3025[k_3025].setHorizontalAlignment(SwingConstants.CENTER);
117         panelArray_3025.add(labelArray_3025[k_3025]);
118     }
119
120     panelArray_3025.revalidate();
121     panelArray_3025.repaint();
122 }
```

14. Kode ini menjalankan algoritma Bubble Sort secara bertahap (langkah demi langkah) untuk mengurutkan sebuah array dan memperbarui tampilan antarmuka (GUI) secara langsung. Pada setiap pemanggilan, kode akan membandingkan dua elemen array yang bersebelahan, menukarnya jika urutannya salah, mengubah warna elemen tersebut di layar untuk visualisasi, dan mencatat prosesnya ke dalam log teks. Setelah satu putaran perbandingan selesai, indeks akan diatur ulang untuk memulai putaran berikutnya hingga seluruh elemen terurut, di mana sistem kemudian akan menonaktifkan tombol kontrol dan menampilkan pesan bahwa proses pengurutan telah selesai.


```

123 private void performStep() {
124     if (!sorting_3025 || i_3025 >= array_3025.length - 1) {
125         sorting_3025 = false;
126         stepButton_3025.setEnabled(false);
127         JOptionPane.showMessageDialog(this, "Sorting selesai!");
128         return;
129     }
130     resetHighlights();
131     StringBuilder stepLog_3025 = new StringBuilder();
132     labelArray_3025[j_3025].setBackground(Color.CYAN);
133     labelArray_3025[j_3025 + 1].setBackground(Color.CYAN);
134     if (array_3025[j_3025] > array_3025[j_3025 + 1]) {
135         // Swap
136         int temp_3025 = array_3025[j_3025];
137         array_3025[j_3025] = array_3025[j_3025 + 1];
138         array_3025[j_3025 + 1] = temp_3025;
139         labelArray_3025[j_3025].setBackground(Color.RED);
140         labelArray_3025[j_3025 + 1].setBackground(Color.RED);
141         stepLog_3025.append("Langkah ").append(stepCount_3025).append(": Menukar elemen ke-")
142             .append(j_3025).append(" (").append(array_3025[j_3025 + 1]).append(") dengan ke-")
143             .append(j_3025 + 1).append(" (").append(array_3025[j_3025]).append(")\n");
144     } else {
145         stepLog_3025.append("Langkah ").append(stepCount_3025).append(": Tidak ada pertukaran antara ke-")
146             .append(j_3025).append(" dan ke-").append(j_3025 + 1).append("\n");
147     }
148     stepLog_3025.append("Hasil: ").append(arrayToString(array_3025)).append("\n\n");
149     stepArea_3025.append(stepLog_3025.toString());
150     updateLabels();
151     j_3025++;
152     if (j_3025 >= array_3025.length - i_3025 - 1) {
153         j_3025 = 0;
154         i_3025++;
155     }
156     stepCount_3025++;
157     if (i_3025 >= array_3025.length - 1) {
158         sorting_3025 = false;
159         stepButton_3025.setEnabled(false);
160         JOptionPane.showMessageDialog(this, "Sorting selesai!");
161     }
162 }

```

15. Kode ini terdiri dari dua fungsi utama untuk memperbarui antarmuka pengguna (GUI) berbasis Java Swing. Fungsi pertama, `updateLabels()`, menggunakan perulangan untuk mengambil nilai dari sebuah array angka dan menampilkannya sebagai teks pada deretan label (`JLabel`) yang sesuai. Fungsi kedua, `resetHighlights()`, berfungsi untuk mengembalikan tampilan visual label-label tersebut ke kondisi awal dengan mengubah warna latar belakangnya menjadi putih. Secara keseluruhan, kode ini digunakan untuk menyinkronkan data angka ke layar dan mengatur ulang penanda visual pada elemen antarmuka tersebut.

```

163 private void updateLabels() {
164     for (int k_3025 = 0; k_3025 < array_3025.length; k_3025++) {
165         labelArray_3025[k_3025].setText(String.valueOf(array_3025[k_3025]));
166     }
167 }
168
169 private void resetHighlights() {
170     for (JLabel label_3025 : labelArray_3025) {
171         label_3025.setBackground(Color.WHITE);
172     }
173 }
174

```

16. Kode ini merupakan metode `reset()` ini berfungsi untuk mengembalikan aplikasi ke kondisi awal atau keadaan semula. Secara teknis, kode ini mengosongkan semua input dan area teks, menghapus elemen visual pada panel, menonaktifkan tombol interaksi, serta mengatur ulang variabel kontrol dan indeks perhitungan kembali ke nilai dasarnya agar proses dapat dimulai lagi dari awal.

```

173 private void reset() {
174     inputField_3025.setText("");
175     panelArray_3025.removeAll();
176     panelArray_3025.revalidate();
177     panelArray_3025.repaint();
178     stepArea_3025.setText("");
179     stepButton_3025.setEnabled(false);
180     sorting_3025 = false;
181     i_3025 = 0;
182     j_3025 = 0;
183     stepCount_3025 = 1;
184 }

```

17. Kode ini berfungsi untuk mengubah sebuah array angka (integer) menjadi satu baris teks (string) yang rapi.

```

185 private String arrayToString(int[] arr_3025) {
186     StringBuilder sb_3025 = new StringBuilder();
187     for (int k_3025 = 0; k_3025 < arr_3025.length; k_3025++) {
188         sb_3025.append(arr_3025[k_3025]);
189         if (k_3025 < arr_3025.length - 1) sb_3025.append(", ");
190     }
191     return sb_3025.toString();
192 }

```

18. Kode ini merupakan fungsi utama (main) yang berfungsi sebagai titik awal untuk menjalankan aplikasi antarmuka grafis (GUI) berbasis Swing di Java. Di dalamnya, perintah `SwingUtilities.invokeLater` digunakan untuk memastikan bahwa pembuatan dan penampilan jendela aplikasi bernama `BubbleGUI_2511533025` dilakukan secara aman pada jalur proses yang tepat (Event Dispatch Thread), sehingga aplikasi dapat muncul di layar dengan stabil saat program dijalankan.

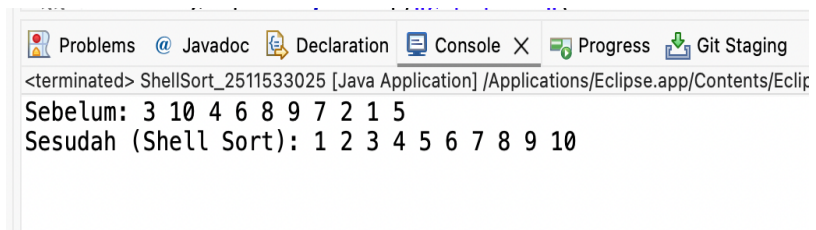
```

193 public static void main(String[] args) {
194     SwingUtilities.invokeLater(() -> {
195         BubbleGUI_2511533025 gui_3025 = new BubbleGUI_2511533025();
196         gui_3025.setVisible(true);
197     });
198 }
199 }

```

f. Hasil projek pekan 8.

1. Output dari Class `ShellSort_2511533025`.

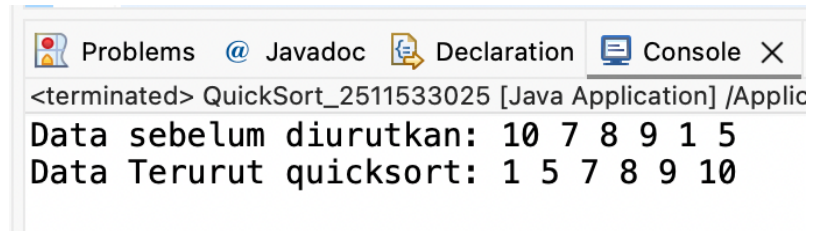


```

Problems  @ Javadoc  Declaration  Console X  Progress  Git Staging
<terminated> ShellSort_2511533025 [Java Application] /Applications/Eclipse.app/Contents/Eclip
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

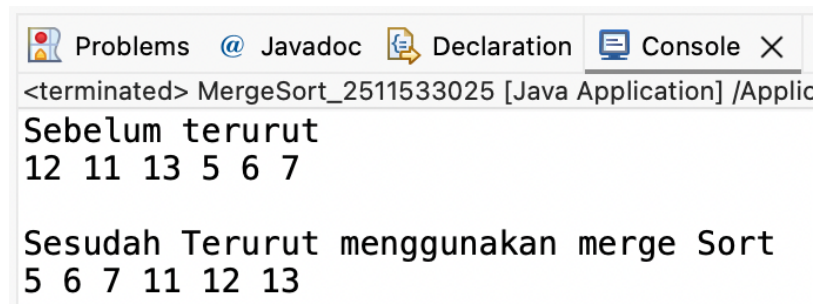
```

2. Output dari Class `QuickSort_2511533025`.



```
Problems @ Javadoc Declaration Console X
<terminated> QuickSort_2511533025 [Java Application] /Applic
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

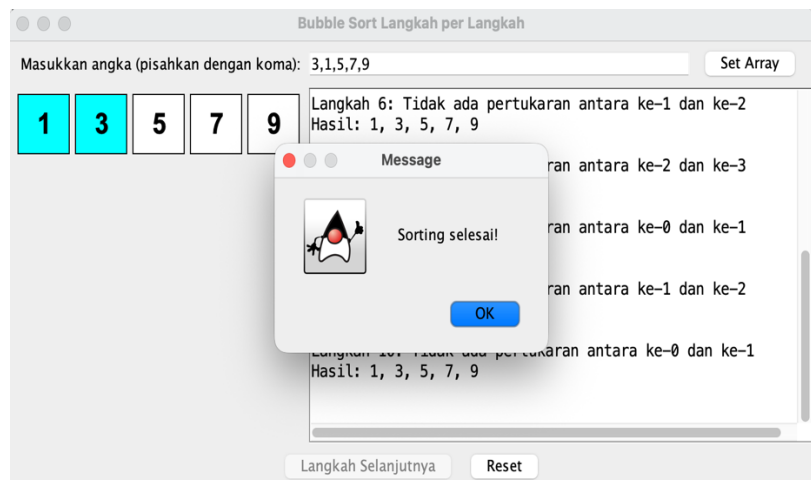
3. Output dari Class MergeSort_2511533025.



```
Problems @ Javadoc Declaration Console X
<terminated> MergeSort_2511533025 [Java Application] /Applic
Sebelum terurut
12 11 13 5 6 7

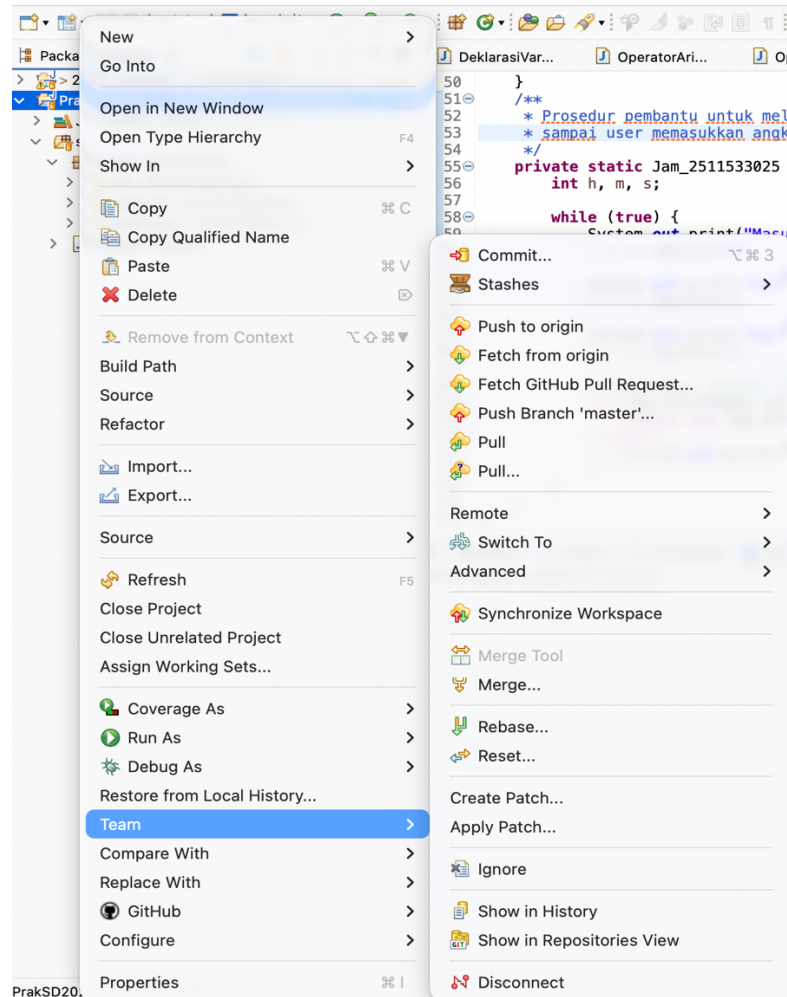
Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13
```

4. Output dari Class BubbleGUI_2511533025.



B. Langkah Penyimpanan

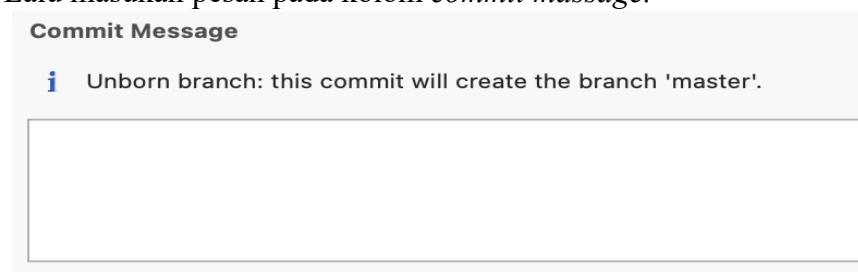
1. Sebelum kita masuk ke langkah, kita harus buat akun github dulu. Jika sudah buat akun github, baru bisa kita simpan di githubnya. Selanjutnya ikutin langkah ini. Tekan kanan mouse pada folder projek, setelah itu pilih team, terus ke *commit*.



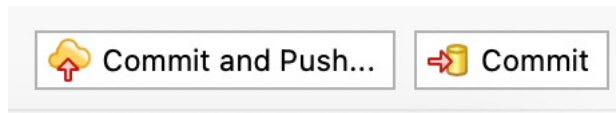
2. Lalu tekan tombol plus 2/double plus warna hijau jika ingin push semua kode, kalau hanya salah satu klik kode yang ingin di push, terus klik tanda plus single.



3. Lalu masukan pesan pada kolom *commit message*.



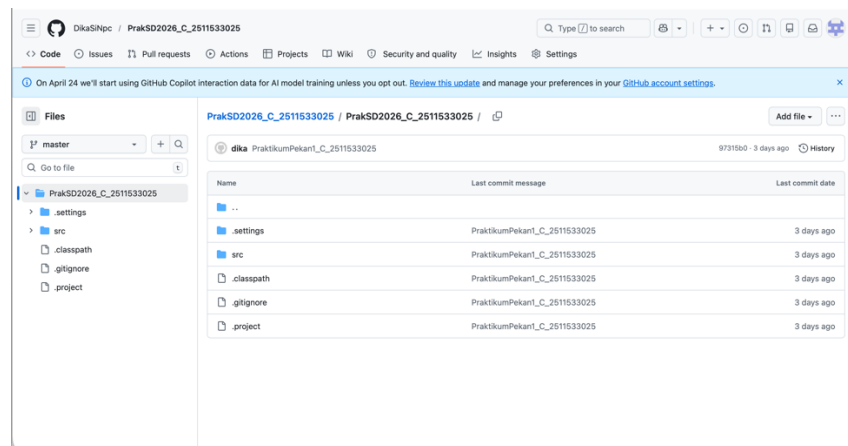
4. Setelah itu tekan *commit* and pus, lalu ikutin arahan untuk memasukan nama dan pw akun git hub kalian, maka projek kalian sudah tersimpan di akun github kalian.



5. Proyek yang sudah disimpan di akun proyek.

 DikaSiNpc/PrakSD2026_C_2511533025

6. Gambar proyek apabila sudah masuk di akun github.



Sedikit tambahan jika ingin cek hasil program nya tekan tombol *run* warna hijau.



Catatan :

Jika kita ingin membuat proyek baru lagi, maka cukup buat folder class yang baru atau package yang baru.

BAB 3 PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa *Shell Sort*, *Quick Sort*, dan *Merge Sort* merupakan algoritma pengurutan lanjut yang menawarkan efisiensi lebih tinggi dibandingkan metode dasar seperti *Bubble Sort*. *Shell Sort* bekerja dengan membandingkan elemen yang memiliki jarak tertentu dan secara iteratif memperkecil *gap* tersebut, *Quick Sort* menggunakan pendekatan *Divide and Conquer* dengan memilih elemen *pivot* untuk mempartisi data, sedangkan *Merge Sort* juga menerapkan *Divide and Conquer* dengan cara membagi data secara rekursif hingga menjadi unit terkecil, lalu menggabungkannya kembali dalam keadaan terurut. Melalui implementasi dalam bahasa pemrograman Java, penulis memahami bahwa *Quick Sort* umumnya memiliki kinerja tercepat untuk data acak dengan kompleksitas waktu rata-rata $O(n \log n)$, namun memiliki kasus terburuk $O(n^2)$, sementara *Merge Sort* menjamin kompleksitas $O(n \log n)$ pada semua kondisi namun membutuhkan memori tambahan untuk proses penggabungan, dan *Shell Sort* berada di antara keduanya.

3.2 Saran

Untuk pengembangan selanjutnya, Disarankan agar praktikan tidak hanya fokus pada kemampuan mengkodekan sintaks algoritma, tetapi juga memperdalam analisis terhadap kompleksitas waktu dan ruang untuk menentukan algoritma yang paling optimal sesuai dengan karakteristik data yang diolah. Praktikum selanjutnya dapat diperkaya dengan studi komparasi kinerja yang mengukur waktu eksekusi nyata dari ketiga algoritma tersebut. Selain itu, pemahaman mengenai implementasi rekursif pada *Quick Sort* dan *Merge Sort* perlu ditingkatkan agar mahasiswa mampu menangani potensi masalah di lapangan.

DAFTAR PUSTAKA

- Sumber daring (website):

[1] W. Suparta, "Pertemuan 6: Pengurutan (Sorting)," INF202: Struktur Data, Universitas Pertamina Jakarta, n.d. [Online]. Tersedia: <https://ocw.upj.ac.id/files/Handout-INF202-INF202-Struktur-Data-Wayan-Pertemuan-6.pdf>. [Diakses: 13 Mei 2026].

[2] D. A. Putri, "Sorting Lanjutan: Merge Sort dan Quick Sort," Universitas Telkom Jakarta, 25 Feb. 2026. [Online]. Tersedia: <https://jakarta.telkomuniversity.ac.id/sorting-lanjutan-merge-sort-dan-quick-sort/>. [Diakses: 13 Mei 2026].

LAPORAN TUGAS PRAKTIKUM PEKAN 8

1. Intruksi Tugas Praktikum Pekan 8

Tema: Mengurutkan Playlist Lagu

1. Deskripsi Tugas

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Metode pengurutan data yang dipilih Shell Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (AZ) atau Durasi (terpendek ke terpanjang).

2. Aturan Penamaan (Wajib)

- Nama Kelas: gunakan NIM lengkap. Contoh: Sorting_2511533025
- Nama variabel & method: wajib pakai 4 digit terakhir NIM. Contoh: dataLagu_3025, shellSort_3025()
- method utama harus shellSort_3025()

3. Struktur Program

Kelas Lagu (sederhana)

Atribut: judul (String), penyanyi (String), durasi (int)

Kelas Sorting_NIM

- Minimal harus ada:
- Array dataLagu_3025 untuk menyimpan maksimal 20 lagu
- Method inputData_3025() – untuk mengisi data awal (minimal 7 lagu)
- Method pilihAlgoritma – cukup implementasikan: shellSort_3025() – urutkan berdasarkan judul A-Z.
- Method tampilData_3025() – menampilkan sebelum dan sesudah sorting

4. Alasan memilih

Tulis di laporan alasan memilih algoritma tersebut yaitu Shell sort (4-5 kalimat).

2. Kode program Class Sorting_2511533025

```
package TugasPekan8_2511533025;
class Lagu_3025 {
    String judul_3025;
    String penyanyi_3025;
    int durasi_3025;
```

```

        Lagu_3025(String judul_3025, String
        penyanyi_3025, int durasi_3025) {
            this.judul_3025 = judul_3025;
            this.penyanyi_3025 = penyanyi_3025;
            this.durasi_3025 = durasi_3025;
        }
    }

    public class Sorting_2511533025 {
        Lagu_3025[] dataLagu_3025 = new Lagu_3025[20];
        int jumlahData_3025 = 0;
        void inputData_3025() {
            dataLagu_3025[jumlahData_3025++] =
                new Lagu_3025("Laskar Pelangi",
                "Nidji", 226);
            dataLagu_3025[jumlahData_3025++] =
                new Lagu_3025("Hati-Hati di Jalan",
                "Muhammad Tulus & Ari Renaldi", 242);
            dataLagu_3025[jumlahData_3025++] =
                new Lagu_3025("Bunga Terakhir", "Bebi
                Romeo", 281);
            dataLagu_3025[jumlahData_3025++] =
                new Lagu_3025("Perfect", "Ed
                Sheeran", 263);
            dataLagu_3025[jumlahData_3025++] =
                new Lagu_3025("Hymn For The Weekend",
                "Coldplay", 258);
            dataLagu_3025[jumlahData_3025++] =
                new Lagu_3025("Believer", "Imagine
                Dragons", 204);
            dataLagu_3025[jumlahData_3025++] =
                new Lagu_3025("Billie", "Michael
                Jackson", 294);
        }
        void tampilData_3025() {
            for (int i_3025 = 0; i_3025 <
            jumlahData_3025; i_3025++) {
                System.out.println((i_3025 + 1) + ". "
                +
                dataLagu_3025[i_3025].judul_3025 + " - "
                +
                dataLagu_3025[i_3025].durasi_3025 + " detik");
            }
        }
        // Shell Sort berdasarkan Judul A-Z
        void shellSort_3025() {
            int n_3025 = jumlahData_3025;
            for (int gap_3025 = n_3025 / 2; gap_3025 > 0;
            gap_3025 /= 2) {
                for (int i_3025 = gap_3025; i_3025 <
                n_3025; i_3025++) {
                    Lagu_3025 temp_3025 =
                    dataLagu_3025[i_3025];
                    int j_3025 = i_3025;
                    while (j_3025 >= gap_3025 &&

```

```

                                dataLagu_3025[j_3025 -
gap_3025].judul_3025
        .compareToIgnoreCase(temp_3025.judul_3025) > 0) {
                                dataLagu_3025[j_3025] =
                                dataLagu_3025[j_3025 -
gap_3025];
                                j_3025 -= gap_3025;
                                }
                                dataLagu_3025[j_3025] = temp_3025;
                                }
                                }
        }
    public static void main(String[] args) {
        Sorting_2511533025 playlist_3025 =
            new Sorting_2511533025();
        playlist_3025.inputData_3025();
        System.out.println("=== Sorting Playlist NIM:
2511533025 ===\n");

        System.out.println("=== Playlist Sebelum
Sorting ===");
        playlist_3025.tampilData_3025();

        playlist_3025.shellSort_3025();

        System.out.println("\n=== Playlist Setelah
Shell Sort (Judul A-Z) ===");
        playlist_3025.tampilData_3025();
    }
}

```

3. Gambar kode program dan output program.

1. Gambar Class Sorting_2511533025

```

1 package TugasPekan8_2511533025;
2 class Lagu_3025 {
3     String judul_3025;
4     String penyanyi_3025;
5     int durasi_3025;
6
7     Lagu_3025(String judul_3025, String penyanyi_3025, int durasi_3025) {
8         this.judul_3025 = judul_3025;
9         this.penyanyi_3025 = penyanyi_3025;
10        this.durasi_3025 = durasi_3025;
11    }
12 }
13
14 public class Sorting_2511533025 {
15     Lagu_3025[] dataLagu_3025 = new Lagu_3025[20];
16     int jumlahData_3025 = 0;
17     void inputData_3025() {
18         dataLagu_3025[jumlahData_3025++] =
19             new Lagu_3025("Laskar Pelangi", "Nidji", 226);
20         dataLagu_3025[jumlahData_3025++] =
21             new Lagu_3025("Hati-Hati di Jalan", "Muhammad Tulus & Ari Renaldi", 242);
22         dataLagu_3025[jumlahData_3025++] =
23             new Lagu_3025("Bunga Terakhir", "Bebi Romeo", 281);
24         dataLagu_3025[jumlahData_3025++] =
25             new Lagu_3025("Perfect", "Ed Sheeran", 263);
26         dataLagu_3025[jumlahData_3025++] =
27             new Lagu_3025("Hymn For The Weekend", "Coldplay", 258);
28         dataLagu_3025[jumlahData_3025++] =
29             new Lagu_3025("Believer", "Imagine Dragons", 204);
30         dataLagu_3025[jumlahData_3025++] =
31             new Lagu_3025("Billie", "Michael Jackson", 294);
32     }
33     void tampilData_3025() {
34         for (int i_3025 = 0; i_3025 < jumlahData_3025; i_3025++) {
35             System.out.println((i_3025 + 1) + ". " +
36                 dataLagu_3025[i_3025].judul_3025 + " - " +
37                 dataLagu_3025[i_3025].durasi_3025 + " detik");
38         }
39     }
40     // Shell Sort berdasarkan Judul A-Z
41     void shellSort_3025() {
42         int n_3025 = jumlahData_3025;
43         for (int gap_3025 = n_3025 / 2; gap_3025 > 0; gap_3025 /= 2) {
44             for (int i_3025 = gap_3025; i_3025 < n_3025; i_3025++) {
45                 Lagu_3025 temp_3025 = dataLagu_3025[i_3025];
46                 int j_3025 = i_3025;
47                 while (j_3025 >= gap_3025 &&
48                     dataLagu_3025[j_3025 - gap_3025].judul_3025
49                     .compareToIgnoreCase(temp_3025.judul_3025) > 0) {
50                     dataLagu_3025[j_3025] =
51                         dataLagu_3025[j_3025 - gap_3025];
52                     j_3025 -= gap_3025;
53                 }
54                 dataLagu_3025[j_3025] = temp_3025;
55             }
56         }
57     }
58     public static void main(String[] args) {
59         Sorting_2511533025 playlist_3025 =
60             new Sorting_2511533025();
61         playlist_3025.inputData_3025();
62         System.out.println("== Sorting Playlist NIM: 2511533025 ==\n");
63         System.out.println("== Playlist Sebelum Sorting ==");
64         playlist_3025.tampilData_3025();
65
66         playlist_3025.shellSort_3025();
67
68         System.out.println("\n== Playlist Setelah Shell Sort (Judul A-Z) ==");
69         playlist_3025.tampilData_3025();
70     }
71 }

```

2. Gambar output kode program



```
<terminated> Sorting_2511533025 [Java Application] /Applications/Eclipse
=== Sorting Playlist NIM: 2511533025 ===

=== Playlist Sebelum Sorting ===
1. Laskar Pelangi - 226 detik
2. Hati-Hati di Jalan - 242 detik
3. Bunga Terakhir - 281 detik
4. Perfect - 263 detik
5. Hymn For The Weekend - 258 detik
6. Believer - 204 detik
7. Billie - 294 detik

=== Playlist Setelah Shell Sort (Judul A-Z) ===
1. Believer - 204 detik
2. Billie - 294 detik
3. Bunga Terakhir - 281 detik
4. Hati-Hati di Jalan - 242 detik
5. Hymn For The Weekend - 258 detik
6. Laskar Pelangi - 226 detik
7. Perfect - 263 detik
```

4. Alasan memilih metode ShellSort karena memiliki proses pengurutan yang lebih cepat dibandingkan Insertion Sort pada data berukuran sedang. Algoritma ini bekerja dengan membandingkan elemen yang berjarak tertentu (gap), sehingga perpindahan data menjadi lebih efisien. Selain itu, implementasi Shell Sort relatif sederhana dan mudah dipahami oleh mahasiswa. Penggunaan memori tambahan juga sangat kecil karena pengurutan dilakukan langsung pada array yang ada. Oleh karena itu, Shell Sort cocok digunakan untuk mengurutkan data playlist lagu berdasarkan judul.

5. Penjelasan sederhana tentang kode program

Program di atas digunakan untuk mengurutkan data playlist lagu menggunakan algoritma Shell Sort. Program terdiri dari dua kelas, yaitu kelas Lagu dan kelas Sorting_2511533025. Kelas Lagu berfungsi sebagai tempat penyimpanan informasi setiap lagu yang meliputi judul, penyanyi, dan durasi. Sementara itu, kelas Sorting_2511533025 berfungsi untuk menyimpan data lagu dalam array, mengisi data lagu, menampilkan data, dan melakukan proses pengurutan.

Pada method inputData_3025(), program mengisi data awal playlist sebanyak tujuh lagu ke dalam array dataLagu_3025. Data yang dimasukkan berupa judul lagu, nama penyanyi, dan durasi lagu dalam satuan detik. Method tampilData_3025() digunakan untuk menampilkan seluruh data lagu yang ada di dalam array. Dengan

method ini, pengguna dapat melihat kondisi playlist sebelum dan sesudah dilakukan proses pengurutan.

Proses pengurutan dilakukan pada method `shellSort_3025()`. Algoritma Shell Sort bekerja dengan membandingkan data yang memiliki jarak tertentu (gap), kemudian jarak tersebut diperkecil secara bertahap hingga menjadi satu. Pada program ini, pengurutan dilakukan berdasarkan judul lagu secara alfabetis dari A sampai Z menggunakan fungsi `compareTolgnoreCase()`. Setelah proses sorting selesai, method `main()` akan menampilkan kembali data playlist sehingga pengguna dapat melihat perubahan urutan lagu dari kondisi awal menjadi urutan yang sudah terurut berdasarkan judul.